

7. 일관성 있는 프롬프트 설계 및 자동화 평가 실습

1. 프롬프트 설계 전략

프롬프트를 어떻게 작성하느냐에 따라 AI 모델의 응답 품질과 **일관성**이 크게 좌우됩니다. 우선 프롬프트를 목적에 따라 **설명형**, **요약형**, **질의응답형** 세 가지로 나누어 보고, 각 유형에서 일관성을 높이기 위한 설계 전략을 알아보겠습니다.

1.1. 설명형 프롬프트

설명형 프롬프트는 특정 개념이나 주제에 대해 자세히 설명을 요청하는 형태입니다. 이 유형의 프롬프트를 설계할 때는 **맥락과 구체적인 요구사항**을 명확히 전달하는 것이 중요합니다. 예를 들어:

Explain how Python's garbage collection mechanism works in simple terms.

위 프롬프트는 Python의 가비지 컬렉션 메커니즘을 간단히 설명해달라는 요청입니다. 설명형 프롬프트의 설계 원칙은 다음과 같습니다:

- **명확한 대상 개념**: 설명을 원하는 대상(예: Python의 가비지 컬렉션)을 프롬프트에 분명히 명시합니다.
- **요구 사항 상세화**: 어떤 수준이나 스타일로 설명을 원하는지 덧붙입니다. 위 예시는 **"in simple terms"**를 포함하여 쉬운 용어로 설명하도록 했습니다. 필요에 따라 **"비유를 들어 설명해줘"** 또는 **"5살 어린이에게 이야기하듯 설명해줘"**처럼 구체적인 톤이나 대상 수준을 지정할 수 있습니다.
- **일관성 유지 요소**: 설명형 응답에서 일관성을 높이려면, 출력 형식이나 다루어야 할 핵심 포인트를 지정할 수 있습니다. 예를 들어 **"3단계로 나누어 설명해줘"** 라고 하면, 모델은 항상 3단계 구조로 답하려고 시도하므로 형식이 일정해집니다.

예시 프롬프트:

"당신은 소프트웨어 교사입니다. 파이썬의 garbage collection(가비지 컬렉션) 방식에 대해 초보자도 이해하기 쉽게 3가지 핵심 포인트로 설명해 주세요."

이렇게 역할과 형식(3가지 포인트)을 지정하면, 모델이 보다 **일관된 구조**로 답을 제공할 가능성이 높아집니다.

1.2. 요약형 프롬프트

요약형 프롬프트는 주어진 텍스트나 정보를 간결하게 정리해 달라는 요청입니다. 요약 작업에서는 **중요한 내용이 빠지지 않고 포함되는 것**이 중요하며, 응답의 **형식 (예: 문장 수나 bullet point 사용)**을 지정함으로써 일관성을 높일 수 있습니다.

- **컨텍스트 제공**: 요약할 원본 텍스트나 정보가 있다면 프롬프트에 충분히 제공해야 합니다. (예: 문서 내용, 기사 전문 등)
- **요약 범위/길이 지정**: 원하는 요약 길이를 명시하면 응답이 과하게 길어지거나 너무 축약되지 않도록 일관성 있게 컨트롤할 수 있습니다. 예를 들어 **"두 문장으로 요약해줘"** 또는 **"한 문단으로 요약해줘"**와 같이 요구합니다.
- **형식 지정**: 목록 형태의 요약을 원하면 **"핵심 내용 3가지를 bullet point로 요약해줘"**라고 명시합니다. 이렇게 하면 모든 응답이 bullet list 형태를 따르게 되어 형식상 일관성을 얻습니다.

예시 프롬프트:

다음 글을 읽고 핵심만 2문장으로 요약해줘:

"Python은 배우기 쉬운 문법과 광범위한 라이브러리 덕분에 ... (중략) ... 이러한 이유로 다양한 분야에서 활용된다."

위 프롬프트에서는 요약 대상 글이 주어져 있고, **"2문장으로 요약"**이라는 분명한 지시가 있으므로 모델은 항상 두 문장으로 핵심을 요약하려고 할 것입니다. 만약 문장 수를 지정하지 않으면 어떤 때는 한 문장, 어떤 때는 세 문장으로 요약되는 등 **출력 길이에 변동성**이 생길 수 있습니다.

1.3. 질의응답형 프롬프트

질의응답형 프롬프트는 사용자의 질문에 대해 정확한 답변을 얻는 형태입니다. 일상적인 Q&A부터 지식 기반 질문까지 다양합니다. 이 유형에서 중요한 점은 **질문의 맥락과 형태**를 명확히 하여 모델이 한결같이 정확한 답변을 내놓도록 유도하는 것입니다.

- **명확한 질문:** 애매모호한 질문은 모델 답변의 일관성을 해칩니다. 예를 들어, "파이썬은 어떻게 사용하나요?"보다는 "파이썬으로 간단한 'Hello World' 프로그램을 작성하는 방법은?"처럼 구체적으로 묻는 것이 좋습니다.
- **필요 시 제약 추가:** 모델이 불필요하게 장황하게 답하거나 엉뚱한 방향으로 나가지 않도록, 원하는 답변 형식을 제시합니다. 예: "한 문장으로 대답해줘" 또는 "표로 정리해줘".
- **맥락 제공:** 질문 자체에 필요한 전제가 있다면 함께 제공합니다. 예: "데이터 분석을 위해 파이썬을 설치하려고 합니다. Anaconda를 활용한 설치 방법을 알려주세요." 처럼요. 이렇게 하면 모델이 일관되게 해당 맥락 내에서 답을 구성하게 됩니다.

예시 프롬프트:

"Q: 파이썬에서 리스트와 튜플의 차이는 무엇인가요?
A: "

위와 같이 Q&A 형식을 직접 프롬프트에 넣어주면 모델이 답변을 바로 이어서 작성하는 형태로 일관성 있게 응답을 생성합니다. 또는 시스템 메시지로 "**당신은 친절한 전문 프로그래밍 Q&A 어시스턴트입니다.**"라는 지시를 주고, 사용자 메시지에 질문을 작성하여 모델이 그 역할에 맞게 답변하도록 유도할 수도 있습니다.

요약하면, **일관성 있는 프롬프트 설계 전략**의 핵심은:

- 요청을 구체적이고 명확하게 작성 (모델이 해석을 덜 하도록)
- 필요한 경우 출력 형식, 길이, 포함해야 할 요소를 지정
- 모델의 역할이나 구체적인 구성을 지시하여 응답의 구조를 예측 가능하게 만들기

이런 전략을 통해 동일한 프롬프트에 대해서도 모델이 가능한 한 **일정한 내용과 형식으로 답변**하도록 유도할 수 있습니다. 다음으로는 실제로 이러한 프롬프트를 사용해 모델을 호출하고, 응답의 일관성과 변동성을 측정하는 실습을 진행해보겠습니다.

2. OpenAI API 활용 실습 준비

이제 OpenAI API를 활용하여 직접 프롬프트를 테스트해보겠습니다. Google Colab 환경에서 실습하므로, 먼저 OpenAI API 파이썬 라이브러리를 설치 및 설정하고, GPT-4o-mini 모델을 호출하기 위한 준비를 합니다.

2.1. OpenAI 라이브러리 설치 및 API 키 설정

Colab 노트북에 OpenAI 패키지가 설치되어 있지 않다면 먼저 설치합니다. 그리고 OpenAI API 키를 Colab에 미리 저장해 두었다면 (`google.colab import userdata` 를 통해) 이를 불러와서 사용합니다.

##실습 7.1. Google Colab 실습

```
# 한글 폰트 설치
!sudo apt-get install -y fonts-nanum
# 폰트 설치 후 Colab 메뉴에서 런타임/세션 다시 시작을 선택합니다.

!pip install -U openai

from google.colab import userdata
import os
import openai

# Colab에 저장한 Secret에서 API 키 가져와 환경 변수에 등록
os.environ["OPENAI_API_KEY"] = userdata.get('OPENAI_API_KEY')

client = openai.OpenAI()
```

- `openai` 1.xx 버전에서는 `OpenAI` 클래스를 사용해 클라이언트를 생성할 수 있습니다. (`timeout` 은 선택적인 파라미터로 응답 대기 시간 설정)

- `userdata.get('OPENAI_API_KEY')` 는 Colab에서 **런타임 > 사용자 정보**에 저장한 API 키를 가져오는 방법입니다. 사전에 해당 키를 Colab에 추가해두어야 작동합니다.
- 이렇게 생성한 `client` 객체를 통해 API를 호출하게 됩니다. 기본 모델을 지정하지 않았다면 이후 호출 시 `model` 파라미터로 명시적으로 **"GPT-4o-mini"**를 사용할 것입니다. (필요에 따라 **"GPT-4o"** 모델명을 지정해 더 성능 좋은 모델을 사용할 수도 있습니다.)

2.2. 예시 프롬프트 정의 및 단일 호출 테스트

이제 실습에 사용할 **예시 프롬프트**를 정의하겠습니다. 앞서 설계 전략을 논의한 내용을 실제로 적용해보겠습니다. 예를 들어, **"Python이 인기 있는 이유 3가지를 bullet point로 나열해줘."** 라는 설명형 프롬프트를 사용하겠습니다. 이 프롬프트의 의도는 Python 언어의 인기 요인을 3가지로 정리하라는 것이며, bullet point 형식을 명시하여 응답 형식의 일관성을 확보하려는 것입니다.

##실습 7.2. Google Colab 설명형 프롬프트 실습

```
prompt = "Python은 인기 있는 프로그래밍 언어입니다. Python이 인기 있는 핵심 이유 3가지를 bullet point로 나열해 주세요."
response = client.chat.completions.create(
    messages=[
        {"role": "user", "content": prompt}
    ],
    model="gpt-4o-mini",
    temperature=0.5
)
print(response.choices[0].message.content)
```

- `prompt` 변수에 한국어로 질문을 작성했습니다. (물론 영어로 작성해도 무방합니다. 모델은 다국어 이해가 가능하므로 선택입니다. 여기서는 한국어 수강자를 고려해 한국어 프롬프트를 사용했습니다.)
- `client.chat.completions.create()` 메서드를 통해 ChatCompletion API를 호출합니다. `model` 파라미터로 **"GPT-4o-mini"**를 지정하여 해당 모델을 사용합니다. `temperature=0.5` 는 중간 정도의 무작위성을 주었습니다. (0에 가까우면 보수적인 일관된 답, 1에 가까우면 창의적이고 다양성 있는 답을 생성)
- 응답 객체의 `choices[0].message.content` 에 모델의 답변 텍스트가 들어있으므로 이를 출력합니다.

위 코드를 실행하면 실제로 GPT-4o-mini 모델이 Python의 인기 요인 3가지를 bullet point 목록으로 답변할 것입니다. **예상 출력** (모델의 응답 예시는 아래와 비슷하게 나올 수 있음):

- 쉬운 문법과 가독성: Python은 코드가 간결하고 읽기 쉬워 초보자도 빠르게 배울 수 있습니다.
- 풍부한 라이브러리와 프레임워크: 데이터 분석, 웹 개발 등을 위한 다양한 라이브러리를 제공하여 개발 생산성을 높입니다.
- 다양한 활용 분야: 웹 개발, 인공지능, 자동화 등 여러 분야에서 활용될 만큼 범용성이 높습니다.

모델 응답은 위와 같이 3개의 bullet point로 구성되었을 것으로 기대합니다. 이제 본격적으로 **동일 프롬프트 반복 호출 실험**을 통해 응답의 일관성과 변동성을 확인해보겠습니다.

2.3. 동일 프롬프트 반복 호출하여 응답 비교

앞서 정의한 `prompt`에 대해 **여러 번** API를 호출하여 결과를 비교해보겠습니다. 이때 한 번 호출할 때마다 모델이 약간씩 다른 출력을 낼 수도 있기 때문에, 여러 응답을 수집하면 **어떤 부분이 일관되고 어떤 부분이 변하는지** 관찰할 수 있습니다.

우선 **같은 조건**에서 반복 호출을 해보겠습니다. `temperature=0.5` 정도로 설정하여 5번 호출하고 결과를 저장해봅시다:

##실습 7.3. Google Colab 반복 호출에 대한 응답 비교 실습

```
# 동일 프롬프트에 대해 여러 번 호출하여 응답 수집
import time

n_runs = 5
```

```

responses = []
for i in range(n_runs):
    response = client.chat.completions.create(
        messages=[
            {"role": "user", "content": prompt}
        ],
        model="gpt-4o-mini",
        temperature=0.5
    )
    text = response.choices[0].message.content
    responses.append(text)
    print(f"=== 응답 {i+1} ===\n{text}\n")
    time.sleep(1) # API 호출 간 1초 휴식 (rate limit 고려)

```

위 코드에서는 동일한 `prompt` 를 5회 호출하고 각 응답 내용을 리스트 `responses` 에 누적했습니다. 또한 각 응답을 콘솔에 출력하여 눈으로도 차이를 확인할 수 있게 했습니다. (실제 Colab에서 실행 시, 응답 간에 약간의 지연을 두는 것이 API 이용 상 안전합니다.)

예상 관찰 결과: 출력된 5개의 응답은 모두 "Python의 인기 요인 3가지"라는 질문에 답하고 있으므로 기본적으로 비슷한 주제의 내용을 담을 것입니다. 대부분 "**쉬운 문법**", "**풍부한 라이브러리**", "**다양한 활용 분야**" 와 같이 유사한 요점들이 언급될 가능성이 높습니다. 그러나 표현 방식이나 세부 내용에서 약간씩 차이가 있을 수 있습니다. 어떤 응답은 "대규모 커뮤니티 지원"을 언급할 수도 있고, 문장의 길이나 어조가 다를 수도 있습니다.

우리가 간단히 눈으로 비교해도 볼 수 있지만, 다음 단계에서는 이 차이를 **자동으로 측정**해보겠습니다.

2.4. 매개변수에 따른 응답 변화 실험(Temperature 조절)

모델 응답의 **변동성(랜덤성)**은 주로 `temperature` 파라미터로 제어됩니다. `temperature` 를 낮게 (0에 가깝게) 설정하면 매번 거의 동일한 응답을 얻게 되고, 높게 (1에 가깝게) 하면 응답의 다양성이 커집니다.

이번에는 `temperature` 값을 달리하여 동일 프롬프트에 대한 응답 차이를 비교해보겠습니다. 예를 들어 `temperature=0.2` (낮은 값)일 때와 `temperature=0.8` (높은 값)일 때 각각 5번씩 응답을 받아보겠습니다:

##실습 7.4. Google Colab 매개변수에 따른 응답 변화 실습

```

def get_responses(prompt, temp, runs=5):
    result_texts = []
    print(f'temperature: {temp}')
    for i in range(runs):
        print(f'run #: {i+1}')
        response = client.chat.completions.create(
            messages=[
                {"role": "user", "content": prompt}
            ],
            model="gpt-4o-mini",
            temperature=temp
        )
        result_texts.append(response.choices[0].message.content)
        time.sleep(1) # API 호출 간 1초 휴식 (rate limit 고려)
    return result_texts

# 낮은 변동성 (temperature=0.2)으로 5회 호출
responses_low = get_responses(prompt, temp=0.2, runs=5)
# 높은 변동성 (temperature=0.8)으로 5회 호출
responses_high = get_responses(prompt, temp=0.8, runs=5)

```

```
print("응답(temperature=0.2) 예시:\n", responses_low[0], "\n")
print("응답(temperature=0.8) 예시:\n", responses_high[0])
```

- `get_responses` 함수: 주어진 프롬프트를 `runs` 횟수만큼 호출하여 응답 텍스트들을 리스트로 반환하도록 작성했습니다. 재사용하기 위해 함수로 정의했습니다.
- `responses_low` 에는 temperature 0.2에서의 응답 5개 리스트가, `responses_high` 에는 0.8에서의 응답 5개 리스트가 저장됩니다.
- 각각 첫 번째 응답 예시를 출력하여 두 설정 간의 응답 차이를 감각적으로 확인해봅니다.

예상 결과 해석:

- `temperature=0.2` 의 경우: 5개의 응답이 거의 **유사한 내용**을 담고 있을 것입니다. 랜덤성이 낮으므로 모델은 가장 가능성이 높은 답변 패턴을 일관되게 반복합니다. 아마도 5번 모두 "문법이 쉬움", "라이브러리 풍부", "다양한 활용" 이 세 가지를 꼽을 확률이 높습니다. 문장 표현도 거의 비슷하게 나올 수 있습니다.
- `temperature=0.8` 의 경우: 5개의 응답이 **다양한 표현**을 보일 것입니다. 어떤 응답은 앞의 세 가지 이유를 언급하되 표현을 달리 하고, 가끔은 "대규모 커뮤니티"나 "오픈 소스 생태계" 같은 이유를 추가로 언급하거나 교체할 수 있습니다. 예를 들어 한 번은 "대규모 커뮤니티 지원"을 3가지 중 하나로 꼽고, 다른 번에는 대신 "유연한 동적 타이핑"을 언급하는 식으로 **포인트 자체의 변동**이 생길 수 있습니다. 또한 문장의 길이나 어조도 어떤 때는 간결하게, 어떤 때는 자세히 설명하는 차이 등이 나타날 수 있습니다.

이처럼 `temperature` 조절을 통해 **응답의 일관성 vs 창의성**을 선택할 수 있습니다. 이제, 수집한 응답들을 정량적으로 분석해보겠습니다.

2.5. 결과 수집 및 데이터프레임 정리

이제 두 시나리오(temperature 0.2와 0.8)에 대해 얻은 응답 리스트를 가지고, 각 응답의 특성을 정리해보겠습니다. 분석하고자 하는 항목들은 다음과 같습니다:

- **응답 길이:** 각 응답의 단어 수 또는 글자 수 (텍스트 길이) - 일관된 프롬프트에 대해 길이가 얼마나 달라지는지 확인
- **핵심 키워드 포함 여부:** "문법(syntax)", "라이브러리(library)", "다양/범용(versatile)", "커뮤니티(community)"와 같이 Python 인기 요인으로 예상되는 핵심어들이 각 응답에 포함되었는지 여부
- **형식 준수 여부:** 요청대로 3가지 bullet point로 답했는지 (항목 개수 확인)

먼저, 수집한 응답들을 Pandas 데이터프레임으로 정리해보겠습니다. 응답 텍스트 전체를 다 넣으면 표가 복잡해지므로, 여기서는 **메트릭 중심**으로 컬럼을 구성하겠습니다.

##실습 7.5. Google Colab 결과 수집 및 데이터프레임 정리 실습

```
import pandas as pd

# 분석하고자 하는 키워드 리스트
keywords = ["문법", "직관적", "라이브러리", "커뮤니티"]

def analyze_responses(responses):
    """응답 리스트를 받아 각 응답의 길이와 키워드 포함 여부 등을 분석"""
    data = []
    for i, text in enumerate(responses, start=1):
        # 단어 수 계산 (공백 기준 분할)
        word_count = len(text.split())
        # 키워드 포함 여부 딕셔너리
        presence = {kw: (kw in text) for kw in keywords}
        # bullet 포인트 개수 세기
        bullet_count = text.count('-') # 항목 수
        data.append({
            "run": i,
            "word_count": word_count,
```

```

        "bullets": bullet_count,
        **presence
    })
    return pd.DataFrame(data)

```

```

df_low = analyze_responses(responses_low)
df_high = analyze_responses(responses_high)

```

```

print("Temperature 0.2 응답 분석 결과:")
display(df_low) # Colab에서 데이터프레임 출력 (혹은 print(df_low.to_markdown()) 텍스트 출력 가능)

```

```

print("\nTemperature 0.8 응답 분석 결과:")
display(df_high)

```

- **keywords** 리스트에는 한국어 키워드를 넣었습니다. (모델 출력이 한국어이므로 한국어 키워드로 확인. 만약 영어로 프롬프트를 썼다면 "syntax", "library", "versatile", "community" 등으로 설정하면 됩니다.)
- **analyze_responses** 함수는 각 응답 텍스트에 대해 단어 수 (**word_count**), **키워드 포함 여부** (**True/False**), 그리고 **bullet** 형식으로 답했을 때 항목 수 (**bullets**)를 추출하여 딕셔너리로 만들고, 이러한 딕셔너리들의 리스트를 데이터프레임으로 변환합니다.
 - **text.split()** 으로 단순히 단어 수를 센 것은 띄어쓰기 기준이라 정확한 토큰 수는 아니지만, 비교 목적으로 충분합니다.
 - **presence** 딕셔너리는 각 키워드에 대해 해당 텍스트에 그 키워드(문자열)가 포함되어 있는지를 True/False로 나타냅니다.
 - **bullet_count** 는 응답 텍스트의 개행(newline) 수를 세어 bullet 목록 항목 개수를 추정했습니다. (bullet 목록은 각 항목이 줄 바꿈으로 구분되므로, 줄바꿈 수+1을 항목 개수로 봤습니다. 완벽한 방법은 아니지만, 현재 프롬프트는 항상 3개 항목이라 큰 무리가 없습니다.)

예상 데이터프레임 (Temperature 0.2):

run	word_count	bullets	문법	라이브러리	다양	커뮤니티
1	15	3	True	True	True	False
2	15	3	True	True	True	False
3	16	3	True	True	True	False
4	15	3	True	True	True	False
5	15	3	True	True	True	False

(실제 값은 모델 응답에 따라 다르지만, 대체로 이런 패턴을 예상합니다. 모든 응답에서 "문법", "라이브러리", "다양"은 언급되고 "커뮤니티"는 언급되지 않은 상황 가정.)

예상 데이터프레임 (Temperature 0.8):

run	word_count	bullets	문법	라이브러리	다양	커뮤니티
1	17	3	True	True	True	False
2	22	3	True	True	False	True
3	18	3	True	True	True	False
4	25	3	True	True	True	True
5	16	3	True	True	False	False

(마찬가지로 예시입니다. 몇몇 응답에서 "커뮤니티"를 언급하여 True가 있고, 어떤 응답에서는 "다양" 관련 키워드가 빠진 경우도 보입니다.)

이제 이러한 표를 통해 응답 일관성을 평가해보겠습니다.

2.6. 응답 일관성 자동 평가

이 단계에서는 데이터프레임에 정리된 정보를 활용하여, **정량적인 평가 지표**를 도출합니다. 우리가 관심 갖는 일관성 요소별로 몇 가지 지표를 계산해볼 수 있습니다:

- **길이 일관성:** 응답 길이의 평균과 표준편차 또는 범위를 확인합니다. (예: 모든 응답이 15 ± 1 단어로 거의 동일하다면 길이 일관성이 높음)
- **형식 일관성:** bullet 목록의 항목 수가 모두 3인지 확인합니다. 요청한 형식을 모두 준수했다면 형식 일관성 100%로 볼 수 있습니다.
- **내용 일관성 (핵심 키워드 기준):** 미리 정한 핵심 키워드들이 모든 응답에 등장했는지 비율을 계산합니다. 예를 들어 4개의 핵심 키워드 중 3개가 모든 응답에 들어있다면 75% 일관성. 또는, 각 키워드별로 몇 %의 응답에서 등장했는가를 측정할 수 있습니다.

우선 길이와 형식부터 보겠습니다. `df_low`와 `df_high`를 이용해 간단히 Python 코드로 계산하면:

##실습 7.6. Google Colab 응답 길이 일관성 실습

```
# 길이 및 형식 일관성 평가
for label, df in [("Low Temp (0.2)", df_low), ("High Temp (0.8)", df_high)]:
    lengths = df["word_count"]
    bullets = df["bullets"]
    print(f"\n[{label}]\n")
    print(f"- 평균 단어 수: {lengths.mean():.1f}, 표준편차: {lengths.std():.1f}, 범위: {lengths.min()}~{lengths.max()} 단어")
    print(f"- 형식 일관성 (항목 3개로 답변): {(bullets == 3).mean()*100:.0f}% ({(bullets == 3).sum()}/{len(bullets)})")
```

예상되는 결과 해석:

- **Temperature 0.2:** 모든 응답이 거의 동일한 길이 (예시에서는 15 단어 안팎, 표준편차 매우 작음)이고, `bullets`는 5개 모두 3이므로 형식 일관성 100%입니다.
- **Temperature 0.8:** 응답 길이의 변동이 더 큼니다. 위 예시 데이터에서는 최소 16 ~ 최대 25 단어로 범위가 넓고, 표준편차도 더 클 것입니다. 형식은 여전히 모두 3개 bullet을 유지했다면 100%이지만, 만약 어느 응답이 4개를 썼거나 3개 미만으로 답했다면 그 비율만큼 감소했을 겁니다. (우리 예시에서는 prompt 자체에 "3가지"라고 명시했으므로 모두 3으로 나와 형식은 100%로 예상합니다.)

다음으로, **핵심 키워드 포함 여부**를 요약해보겠습니다. 각 키워드별로 True인 응답의 비율을 계산하면 됩니다:

##실습 7.7. Google Colab 응답 내용 일관성 실습

```
keywords = ["문법", "직관적", "라이브러리", "커뮤니티"]
for label, df in [("Low Temp (0.2)", df_low), ("High Temp (0.8)", df_high)]:
    rates = {kw: df[kw].mean()*100 for kw in keywords} # True의 평균*100 = True 비율(%)
    print(f"\n[{label}] 핵심 키워드 포함율:")
    for kw, rate in rates.items():
        print(f"- {kw}: {rate:.0f}%")
```

예상 결과 해석:

- **Temperature 0.2 시나리오:**
 - "문법": 100% (모든 응답에 포함)
 - "라이브러리": 100%
 - "다양": 100%
 - "커뮤니티": 0% (어느 응답에도 언급되지 않음)이 경우 사전에 우리가 핵심이라고 생각한 3가지("문법", "라이브러리", "다양")는 모든 응답에 빠짐없이 등장했으므로, **내용 일관성**이 매우 높다고 할 수 있습니다. 반면 "커뮤니티"는 한 번도 등장하지 않았는데, 이는 프롬프트가 3가지만 꼽

이라고 제한했고 모델이 다른 세 가지를 선택하는 바람에 일관되게 배제된 것으로 볼 수 있습니다 (오히려 이것도 일관성의 한 부분이네요).

- **Temperature 0.8 시나리오:**

- "문법": 100% (대부분 Python 언급하면 빠지지 않는 요소인 듯)
- "라이브러리": 100%
- "다양": 80% (5번 중 4번 등장, 한 번은 다른 요소로 대체되었을 가능성)
- "커뮤니티": 40% (5번 중 2번 등장)

여기서 "문법"과 "라이브러리"는 여전히 모든 응답에 포함되어 높지만, "다양(versatile)"은 한 응답에서 누락되어 80%, "커뮤니티"는 소수 응답에서만 등장하여 40%로 나왔습니다. 즉, 창의성을 높인 시나리오에서는 **주요 특징 두 가지는 일관되게 나오지만, 나머지 한 자리는 가변적으로 다른 특징(예: 커뮤니티 지원 등)으로 대체되는 경향**을 볼 수 있습니다.

이런 수치를 종합하여 **일관성 점수**를 정의해볼 수도 있습니다. 어떻게 정의하느냐에 따라 다르겠지만, 한 가지 예로 "우리가 중요하게 생각한 핵심 키워드 3개 중 얼마나 일관되게 언급되었나"를 점수화해보겠습니다.

예를 들어:

- Temperature 0.2의 경우 핵심 3개 모두 100%이므로 **내용 일관성 100점**.
- Temperature 0.8의 경우 핵심 3개 중 2개는 100%인데 1개가 80%이므로, 단순 평균내면 약 **93점** 정도로 볼 수 있습니다.

다만 "커뮤니티"처럼 추가로 등장하는 요소는 변동성의 표현이므로, 꼭 일관성 점수에 포함할 필요는 없습니다 (필요에 따라 가중치 없이 평균하거나, 핵심 3개만 대상으로 계산). 형식 일관성은 두 경우 모두 100%였으므로 형식 측면에서는 둘 다 만점입니다. 길이 일관성은 0.2 시나리오가 훨씬 높습니다. 만약 길이 편차까지 하나의 지표로 환산하려면 편차가 작은 쪽에 높은 점수를 주는 방식(예: 범위 0이면 100점, 범위가 커질수록 감점) 등을 생각해볼 수 있습니다.

요약하면, **Temperature 0.2 시나리오**는 거의 동일한 답변을 반복해 **응답 일관성**이 매우 높았다고 평가할 수 있고, **Temperature 0.8 시나리오**는 표현의 다양성으로 인해 **일관성은 다소 낮지만 새로운 정보(예: 커뮤니티 언급) 등장** 등의 **다양성**은 높았다고 평가됩니다.

이를 종합한 평가 결과를 정리하면 다음과 같습니다:

- **내용 일관성:** 낮은 temperature에서는 우리가 기대한 핵심 내용들이 모든 응답에 포함되었음 (일치율 100%), 높은 temperature에서는 핵심 내용 23개 중 일부가 다른 내용으로 대체되기도 함 (일치율 약 80~100% 사이).
- **형식 일관성:** 두 시나리오 모두 프롬프트 지시에 따라 3가지 bullet 형식을 정확히 지킴 (100%).
- **길이 일관성:** 낮은 temperature는 모든 응답 길이가 거의 동일 (편차 매우 작음), 높은 temperature는 응답에 따라 길이 편차가 있음 (단어 수 범위 수개의 차이 발생).

이러한 분석을 통해, 프롬프트가 동일해도 생성 파라미터에 따라 응답의 **일관성 vs 다양성** 프로파일이 크게 달라짐을 확인했습니다.

2.7. 시각화 및 결과 해석

##실습 7.8. Google Colab 시각화 실습: temperature 변화에 따른 응답 길이

```
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib.font_manager as fm
import pandas as pd # Import pandas

# 한글 폰트 설정
font_path = '/usr/share/fonts/truetype/nanum/NanumBarunGothic.ttf' # 나눔바른고딕 경로
fontprop = fm.FontProperties(fname=font_path, size=12)

# 데이터 준비
lengths_low = df_low["word_count"] # temperature 0.2 응답 길이
```



```

lengths_high = df_high["word_count"] # temperature 0.8 응답 길이

# Create a DataFrame for seaborn
data = pd.DataFrame({
    "Temperature": ["0.2"] * len(lengths_low) + ["0.8"] * len(lengths_high),
    "Word Count": lengths_low.tolist() + lengths_high.tolist()
})

# seaborn을 사용하여 box plot 생성
plt.figure(figsize=(8, 6)) # 그림 크기 설정
sns.boxplot(x="Temperature", y="Word Count", data=data) # Use the DataFrame

# 제목, 레이블 설정
plt.title("응답 길이 분포 (Temperature 비교)", fontproperties=fontprop)
plt.xlabel("Temperature", fontproperties=fontprop)
plt.ylabel("응답 길이 (단어 수)", fontproperties=fontprop)

# 그래프 출력
plt.show()

```

이제 앞서 계산한 지표들을 시각화하여 한눈에 비교해보겠습니다. 우선 응답 **길이 분포**를 temperature에 따라 상자그림(box plot)으로 나타내 보겠습니다. 좌측은 temperature 0.2, 우측은 0.8에 대한 응답 길이 분포입니다.

응답 길이 분포를 temperature에 따라 보여주는 상자 그림. 위 그래프에서 왼쪽 **Temperature 0.2** 상자그림은 상자와 선이 거의 한 점에 몰려 있는데, 이는 5회 응답의 단어 수가 모두 13~15 단어 사이로 거의 같음을 보여줍니다. 반면 오른쪽 **Temperature 0.8** 상자그림은 상자의 높이가 높고 위아래 선이 길게 그려져 있으며, 일부 응답은 30단어 이상으로 동그라미(이상치)로 표시되었음을 볼 수 있습니다. 이 그래프는 **temperature가 높을수록 응답 길이의 변동이 커지는** 경향을 시각적으로 잘 나타냅니다. 즉, 낮은 temperature에서는 항상 비슷한 길이로 답변했지만, 높은 temperature에서는 어떤 답변은 짧고 어떤 답변은 길어지는 등 길이 **편차가 증가**했습니다.

##실습 7.9. Google Colab 시각화 실습: temperature 변화에 따른 키워드 포함 비율

```

import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib.font_manager as fm
import numpy as np

# 한글 폰트 설정
font_path = '/usr/share/fonts/truetype/nanum/NanumBarunGothic.ttf' # 나눔바른고딕 경로
fontprop = fm.FontProperties(fname=font_path, size=12)

# 키워드 포함 비율 계산
keywords = ["문법", "직관적", "라이브러리", "커뮤니티"]
rates_low = {kw: df_low[kw].sum() for kw in keywords} # Temperature 0.2에서 키워드 등장 횟수
rates_high = {kw: df_high[kw].sum() for kw in keywords} # Temperature 0.8에서 키워드 등장 횟수

# 막대 그래프 생성
bar_width = 0.35 # 막대 너비
index = np.arange(len(keywords)) # 키워드 위치

fig, ax = plt.subplots(figsize=(10, 6)) # Figure 및 Axes 객체 생성

# Temperature 0.2 막대 (노란색)
rects1 = ax.bar(index, rates_low.values(), bar_width, color='gold', label='Temperature 0.2')

```

```
# Temperature 0.8 막대 (주황색)
rects2 = ax.bar(index + bar_width, rates_high.values(), bar_width, color='darkorange', label='Temperature
0.8')

# 제목, 레이블, 범례 설정
ax.set_title('핵심 키워드 포함 비율 (최대 5회 기준)', fontproperties=fontprop)
ax.set_xlabel('키워드', fontproperties=fontprop)
ax.set_ylabel('포함 횟수', fontproperties=fontprop)
ax.set_xticks(index + bar_width / 2) # x축 눈금 위치 조정
ax.set_xticklabels(keywords, fontproperties=fontprop) # x축 눈금 레이블 설정
ax.legend()

plt.show()
```

다음으로, **핵심 키워드 포함 비율**을 시각화한 막대 그래프를 살펴보겠습니다. 아래 그래프에서 각 키워드별로, **노란색 막대**는 Temperature 0.2 시나리오에서 그 키워드가 포함된 응답 수를 나타내고, **주황색 막대**는 Temperature 0.8 시나리오에서의 값을 나타냅니다 (최대 5회 기준).

Temperature 조건에 따른 핵심 키워드 포함 횟수 비교 그래프. 이 그래프에서 좌측 두 키워드 "**문법**"과 "**라이브러리**"의 경우 노란색과 주황색 막대가 모두 5로 동일합니다. 즉, 두 시나리오 모두 5회 응답 내내 이 두 키워드는 빠짐없이 등장했다는 뜻입니다 (100% 일치). "**다양**"의 경우 노란색은 5, 주황색은 4로 나타나는데, 이는 높은 temperature 시나리오에서 한 번 해당 키워드가 누락되었음을 의미합니다. 마지막으로 "**커뮤니티**" 키워드는 노란색 막대가 0, 주황색은 2입니다. 낮은 temperature에서는 단 한 번도 언급되지 않았지만, 높은 temperature에서는 5번 중 2번 등장하여 다양성이 증가한 부분을 보여줍니다.

이 시각화를 종합하면, **Temperature 0.2 (노란색)** 설정은 모델이 **동일한 핵심 답변 요소만 반복**하게 만들어 응답의 내용이 매우 일관적이었고, **Temperature 0.8 (주황색)** 설정은 모델이 가끔씩 다른 요소도 언급하도록 허용하여 응답 내용에 **변화의 폭**이 생겼음을 알 수 있습니다. 다시 말해, temperature 파라미터를 조절함으로써 일관성과 다양성 사이의 트레이드오프를 컨트롤할 수 있음을 확인했습니다.

2.8. CSV/JSON로 결과 저장

마지막으로, 수집된 결과와 분석 내용을 **파일로 저장**하는 방법을 알아보겠습니다. 실험 결과를 CSV나 JSON 형태로 저장해 두면 추후에 다시 분석하거나, 다른 도구로 시각화하거나, 보고서를 작성하는 데 활용할 수 있습니다. Pandas 데이터프레임의 내장 함수인 `to_csv` 와 `to_json` 을 사용하면 손쉽게 저장할 수 있습니다.

##실습 7.10. Google Colab 파일(CSV, JSON) 저장 실습

```
# 데이터프레임을 CSV 파일로 저장
df_high.to_csv('prompt_responses_high.csv', index=False, encoding='utf-8')
df_low.to_csv('prompt_responses_low.csv', index=False, encoding='utf-8')

# 데이터프레임을 JSON 파일로 저장 (records 형식)
df_high.to_json('prompt_responses_high.json', orient='records', force_ascii=False)
df_low.to_json('prompt_responses_low.json', orient='records', force_ascii=False)
```

위 코드 실행 후 Colab 환경의 파일 탐색기(왼쪽 사이드바)에서 `prompt_responses_high.csv` 등의 파일을 확인할 수 있습니다. `files.download('prompt_responses_high.csv')` 와 같은 코드를 사용하면 로컬 PC로 다운로드도 가능합니다. JSON으로 저장할 때 `force_ascii=False` 를 준 것은 한글이 포함되어 있어서 Unicode가 깨지지 않도록 하기 위함입니다.

이렇게 저장된 CSV를 Excel로 열어본다거나, JSON을 다른 어플리케이션에서 불러와서 활용할 수 있습니다. 우리 실험의 경우 두 시나리오(`df_low` , `df_high`)를 따로 저장했지만, 하나의 CSV에 temperature 값까지 포함해서 저장하고 싶다면 데이터프레임을 합친 후 `temperature` 컬럼을 추가해 구분할 수도 있을 것입니다.

3. 실무 적용 방안 및 결론

이번 실습을 통해 **프롬프트 엔지니어링**에서 일관성이라는 측면을 어떻게 관리하고 평가할 수 있는지 배웠습니다. 정리하면 다음과 같습니다:

- **프롬프트 설계:** 모델 답변의 일관성을 높이려면 처음부터 프롬프트에 형식, 내용, 스타일에 대한 제약을 명확히 주는 것이 중요합니다. 실무에서 새로운 프롬프트를 디자인할 때, 우선 작은 예시들로 실험해보고 원하는 형식으로 일관되게 답이 나오는지 확인해야 합니다. 만약 모델의 응답이 들쭉날쭉하면 프롬프트를 개선하거나 추가 지시를 넣어서 교정해야 합니다.
- **파라미터 튜닝:** production 환경에서 응답의 안정성이 중요하다면 temperature 를 낮게 설정하여 매번 비슷한 답변을 얻게 할 수 있습니다. 반대로 답변의 창의성이나 다양한 표현이 중요하다면 temperature 를 높여 더욱 풍부한 응답을 유도할 수 있습니다. 실무에서는 이 값을 적절히 조정하여 서비스의 성격에 맞는 응답을 얻도록 합니다 (예: 고객 지원 챗봇은 일관성이 중요하므로 낮은 temperature, 창의적인 글 생성 앱은 높은 temperature).
- **반복 테스트와 자동 평가:** 하나의 프롬프트를 설정했으면, 우리가 한 것처럼 여러 번 응답을 받아보면서 예상치 못한 변형이나 오류가 없는지 검증하는 과정이 필요합니다. 이를 자동화해두면 프롬프트 수정 -> 테스트 -> 평가 사이클을 빠르게 돌릴 수 있습니다. 예를 들어, 핵심 키워드 체크나 형식 체크 로직을 코드로 만들어 두면, 새로운 프롬프트에 대해서도 쉽게 검증할 수 있습니다. 이 방식은 대규모 프롬프트 튜닝 작업이나 모델 평가 작업에도 응용할 수 있습니다.
- **모델 선택:** 실습에서는 gpt-4o-mini를 주로 사용했지만, 중요한 경우 최종 결과를 gpt-4o와 같은 더 성능 좋은 모델로 확인할 수 있습니다. 작은 모델로 빠르게 아이디어를 실험하고, 마지막에 큰 모델로 한 번씩 검증하는 식입니다. 큰 모델일수록 응답의 품질은 높지만 비용이 크고 속도가 느릴 수 있으므로, 실무에서는 둘을 적절히 병행하여 사용하면 효율적입니다.
- **로그와 모니터링:** 응답의 일관성은 사용자 경험에 직접 영향 주는 요소입니다. 운영 중에는 사용자로부터 수집된 실제 프롬프트-응답 로그를 분석하여, 의도와 다르게 변동성이 큰 경우를 발견하면 프롬프트를 개선하거나 모델 파라미터를 조정하는 피드백 루프를 구축하는 것이 좋습니다. 우리 실험처럼 키워드 출현 빈도나 응답 길이 등의 지표를 모니터링 포인트로 삼을 수 있습니다.

마지막으로, 프롬프트 엔지니어링은 맥락에 따라 일관성과 다양성 간의 균형을 맞추는 작업이라고 할 수 있습니다. 이번 강의 실습을 통해 얻은 통찰을 바탕으로, 여러분은 앞으로 AI 모델에게 원하는 대답을 더 안정적으로 이끌어내고, 필요에 따라서는 모델의 창의적인 활용까지 컨트롤할 수 있게 될 것입니다. 항상 작은 실험과 측정을 병행하여 프롬프트를 다듬어 나가시길 바랍니다.